

C3 - Intelligence pour la robotique: TP8 – Apprentissage par renforcement

Justin Ferdinand

I INTRODUCTION

Ce TP explore l'apprentissage par renforcement en opposition à l'apprentissage supervisé du TP précédent. L'objectif est de permettre au robot Thymio d'apprendre à s'adapter à son environnement en fonction de ses expériences et des interventions humaines, sans nécessiter de données d'entraînement préalables. Cela s'oppose donc à la création d'un jeu de données d'entraînement pour un apprentissage supervisé, et à la nécessité de fournir des sorties cibles à partir d'entrées enregistrées.

II EXERCICE 1 : APPRENTISSAGE DE HEBB

Dans ce premier exercice, j'ajoute une règle de Hebb sur le robot. Le robot modifie ses poids W en fonction de la corrélation entre ses entrées de capteurs de proximité et ses sorties moteurs.

L'équation de mise à jour des poids est donnée par :

$$W_{ij} = W_{ij} + \alpha \cdot y_i \cdot x_j \quad 1.$$

où α est le taux d'apprentissage, y_i l'activité du neurone moteur i et x_j la valeur du capteur de proximité j .

Le code implémenté dans `reward.py` est le suivant :

```
reward.py python
1 #fonction d'activation de type
  Braitenberg
2 def braitenberg(entries, weights,
  bias, activation_gain):
```

```

3         return np.tanh(activation_gain *
4           (np.dot(weights, entries) +
5             bias))
6
7 #fonction de mise à jour des poids
8   selon la règle de Hebb
9
10 def hebb_update(x, y, W, B, alpha):
11     if not np.any(x > 0.02):
12         return
13
14     for j in range(5):
15         xj = x[j]
16         W[0][j] = W[0][j] + alpha *
17           y1 * xj
18         W[1][j] = W[1][j] + alpha *
19           y2 * xj
20
21     B[0] = B[0] + alpha * y1
22     B[1] = B[1] + alpha * y2
```

Ici, j'ai mis un taux d'apprentissage de 0.05 pour que les poids évoluent plus rapidement lors de mes tests.

III EXERCICE 2 : APPRENTISSAGE SUPERVISÉ

Dans cette seconde partie, l'ajout d'un enseignement humain par clavier permet de guider le robot vers le bon comportement selon son environnement. La sortie y est imposée par l'opérateur. La règle de Hebb de la première partie va apprendre l'association entre les entrées et les sorties des moteurs.

Pour stabiliser le comportement, j'ai conservé la structure principale du script et importé trois améliorations :

- activation non linéaire de type Braitenberg (tanh),
- mise à jour Hebb déclenchée une seule fois par appui clavier,
- ralentissement du robot quand la proximité obstacle augmente pour que Thymio ait le temps de réagir.

Le script `reward.py` implémente cette logique :

```
Mode Enseignement python
1 def get_manual_command_from_key(key):
2     if key == Keyboard.UP:
3         return np.array([100.0,
4                           100.0])
5     elif key == Keyboard.DOWN:
6         return np.array([-100.0,
7                           -100.0])
8     elif key == Keyboard.LEFT:
9         return np.array([-100.0,
10                          100.0])
11    elif key == Keyboard.RIGHT:
12        return np.array([100.0,
13                          -100.0])
14    return None
15
16 key_pressed = override
17 is_new_key_press = key_pressed and not prev_key_pressed
18 if is_new_key_press:
19     hebb_update(x, y, W, B, ALPHA)
20 prev_key_pressed = key_pressed
```

Le calcul des commandes moteurs :

```
Contrôle moteur python
1 forward_cmd = (y[0] + y[1]) / 2.0
2 turn_cmd = ((y[1] - y[0]) / 2.0) * TURN_GAIN
3 y = np.array([
4     forward_cmd - turn_cmd,
5     forward_cmd + turn_cmd
```

```
6 ])
7 y = np.clip(y, -100, 100)
8
9 v_left = (y[0] / 100.0) * speed
10 v_right = (y[1] / 100.0) * speed
```

Avec `TURN_GAIN` une valeur qui permet d'adoucir les virages et `speed` une vitesse de base réduite lorsque les capteurs de proximité détectent un obstacle proche.

III.A APPRENDRE À RECULER FACE À UN MUR

Protocole expérimental : saisir THymio, le placer face au mur et reculer avec la touche clavier.

Résultat et observation :

En réalisant cette expérience, j'ai observé que les poids associés aux capteurs de proximité avant ont augmenté, ce qui indique que le robot a appris à associer ces entrées à la sortie de reculer. Le biais appliqué à Thymio a diminué, ce qui lui donne un comportement de peur face à un obstacle. Avec quelques itérations, le robot recule seul face à un mur, même sans intervention. Ce comportement est visible sur la vidéo `mouvement_de_peur.mp4`.

```
1 Poids W:
2 [[-0.061 -0.068 -0.117  0.    0.   ]
3  [-0.134 -0.032  0.017  0.    0.   ]]
```

III.B RÉALISER UN TOUR COMPLET AVEC OBSTACLES

J'ai rencontré beaucoup de difficultés pour réaliser un entraînement de thymio dans le circuit avec obstacles. En effet, le robot a du mal à percevoir les obstacles que je lui donne. Qu'il s'agisse de bouteilles, d'animaux ou des murs ajoutés, les capteurs de proximité indiquent des valeurs nulles alors même que l'obstacle se trouve face à lui. Pour régler ce problème, je pourrais utiliser le LiDAR de Thymio pour l'entraînement,

cela assurerait un meilleur comportement mais cela sort des objectifs de ce TP qui se base sur le véhicule de Braitenberg.

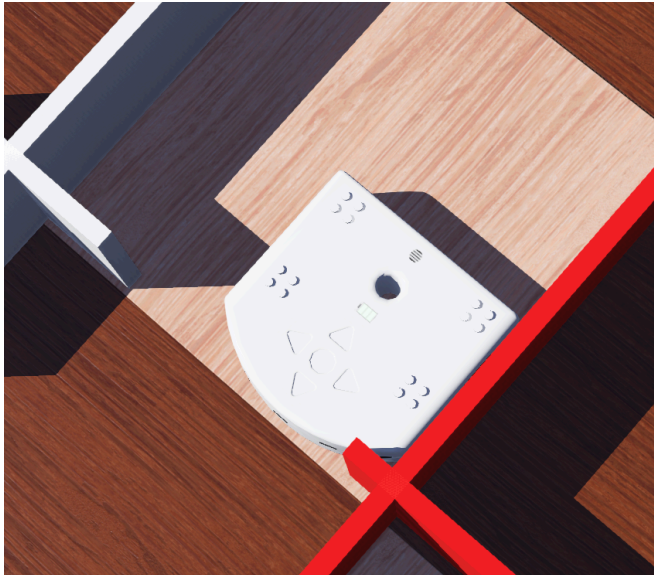


Figure 1: Thymio face à un obstacle (mur) qu'il ne détecte pas.

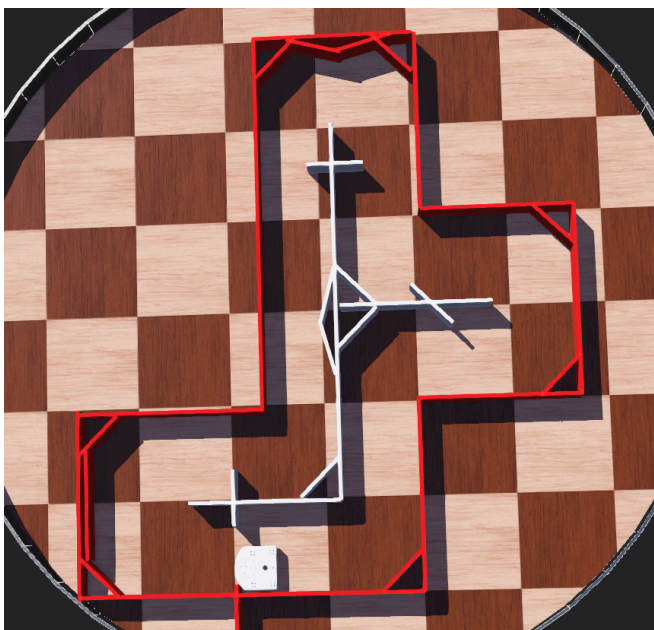


Figure 2: Circuit de test proposé avec obstacles sur la course

III.C RÉALISER UN TOUR COMPLET SANS OBSTACLES

Pour réaliser un tour complet sans obstacles, j'ai placé le robot dans le circuit et j'ai utilisé les touches de direction pour le guider dans les

virages. Après un tour dans chaque sens, le robot a réussi à faire plusieurs tours complets dans les deux sens de manière autonome, en ajustant sa trajectoire en fonction des entrées des capteurs et des virages. Cela montre que Thymio est capable d'apprendre à naviguer très rapidement dans le circuit. Bien évidemment, contrairement à mon implémentation d'un apprentissage supervisé, le robot n'est pas aussi performant dans ses réactions et cela s'explique par un entraînement plus court et moins précis. J'ajoute également que l'apprentissage par renforcement permet de contrer une limitation remarquée lors du TP précédent : Le réseau de neurone ne savait pas comment réagir face à des situations non rencontrées dans le jeu de données d'entraînement, alors que dans ce TP, le robot peut apprendre à réagir à de nouvelles situations au fur et à mesure de son expérience.

La vidéo est disponible dans `tour_complet_sans_obstacles.mp4`.

1	Poids W:
2	$[[0.095 \ -0.047 \ -0.21 \ 0. \ 0. \]]$
3	$[-0.195 \ -0.053 \ 0.11 \ 0. \ 0. \]]$
4	
5	Biais B:
6	$[-0.05 \ -0.095]$

IV CONCLUSION

Pour conclure,

L'apprentissage par renforcement, tout comme l'apprentissage supervisé, ne permet pas ici au réseau de neurones d'apprendre une stratégie globale de parcours du labyrinthe. Le robot apprend surtout des réactions locales à des situations particulières : reculer face à un mur, tourner devant un obstacle, ou corriger sa trajectoire dans un virage. Ces comportements sont utiles, mais restent limités dès que l'environnement devient plus complexe.

Cette limite est renforcée par la qualité de perception des capteurs de proximité de Thymio, qui varie selon la forme et la nature des obstacles. Lorsque les entrées perçues ne reflètent pas correctement la situation réelle, le réseau produit des commandes moins adaptées.

Pour améliorer les performances, une piste concrète serait d'exploiter des capteurs plus fiables, comme le LiDAR ou la vision par caméra. À plus long terme, l'intégration de modèles plus avancés, par exemple des architectures de type VLA (Vision-Language-Action), pourrait offrir une meilleure compréhension du contexte et permettre au robot d'adopter un comportement plus robuste et plus intelligent. J'avais prototypé un bras robotique en stage ayant comme tâche de placer une assiette et des couverts pour montrer que celui-ci faisait preuve de compréhension de sa tâche et de son environnement.